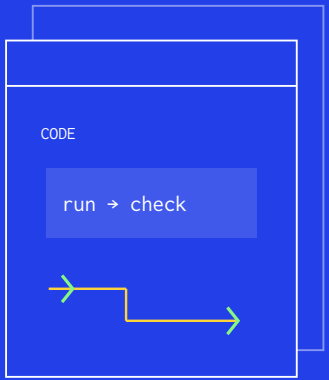
WEEK 3 / LAB 6 / TEAM WORK

MLOps: Machine Learning Operations

Make the pipeline able to fail.

Run your checks in GitHub Actions, write tests that catch real defects, then watch the pipeline fail and recover. Needs Lab 5 merged on main (or develop).



By the end you can show three things

Each one must be visible in your team repository.

1. **Cover**

   Every change going into your integration branches is checked on a clean machine, and the checks cover what matters here, not only code style.

2. **Catch**

   Your tests fail for the defects they claim to catch, and you can show it. One test recomputes a result you recorded earlier.

3. **Repair**

   You saw the pipeline fail for a real reason, found the cause in its logs, and fixed it.

Who does what in your team

The split is yours; every member owns something and reviews something.

EACH MEMBER

Own one part

Pick one: workflow, tests, container job or README. One PR each.

THE WHOLE TEAM

Agree on checks

Decide which checks must be green to merge, and how you set tolerances.

REVIEWER

Try to break it

Open a PR with one believable defect. A check must turn red.

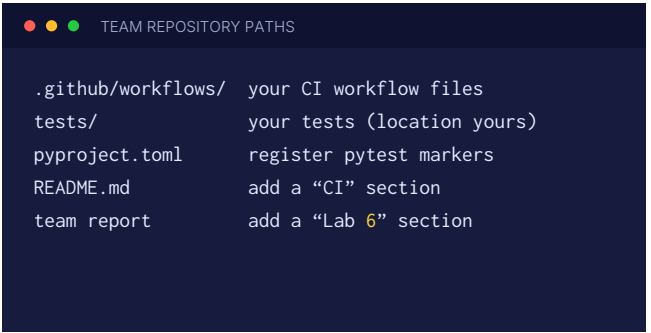
Individual report reports/week-3.md covers Labs 5–6 in your personal repository.

What your team must add

Everything goes in through reviewed pull requests.

REQUIRED

Workflows only run from .github/workflows/ at the repository root. If you add a pytest marker, register it in pyproject.toml, or collection fails.



README: what runs, why, and how to reproduce a failure locally.

Three tools you will use

Lecture 6 explained the ideas. This is what you write or read in this lab.

RUN CHECKS

GitHub Actions

Runs your checks on a clean machine for every pull request.

SELECT TESTS

pytest markers

Labels on tests, so CI runs exactly the group you mean.

READ RESULTS

Checks tab or gh

Open a run, find the first red step, save its log.

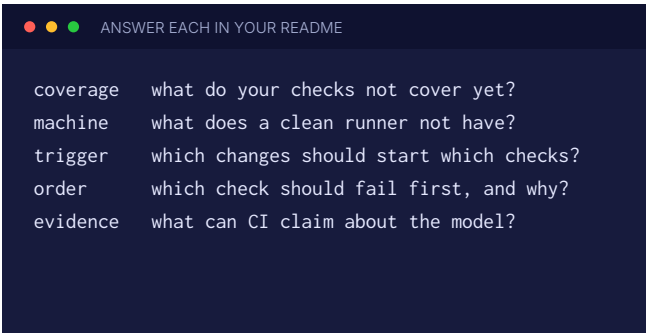
Any tool that gives the same evidence is fine; the evidence matters, not the tool.

Questions your README must answer (1 of 2)

Five questions about what your pipeline checks.

README QUESTIONS

Answer each question in your README: your choice, the option you rejected, and why. Several answers can be good.



Examples and cheat sheets that follow are tools, not answers.

Questions your README must answer (2 of 2)

Five questions about whether your tests work.

README QUESTIONS

A test is worth what it can catch: name the defect, then prove the test can fail.

ANSWER EACH IN YOUR README

```
defects     which bug does each test catch?
failing     can you make each test fail?
tolerance   where does your number come from?
repair      did you fix the cause or hide it?
records     what is still readable in 3 months?
```

Reviewers read your reasons; there is no single correct answer.

Rules that keep CI honest

Fixed for every team; everything else is your choice.

1 **Locked installs only**

Install from the lock file; never regenerate it just to get green. Keep the starter's safe setup: least privilege, no secrets, pinned versions, time limits.

2 **Never mute a failure**

No ignored failures, skipped tests or loosened assertions to get green. The failure you show must be real: a change caused it and a check caught it.

3 **Keep evidence honest**

CI evidence is what the runner itself produced, not your laptop's MLflow runs. No test-period metrics. A tolerance comes from measured variation.

Start from the workflow you already have

Your starter repository already runs checks; read it before you add to it.

READ

Every line

Find the trigger, permissions, runner and steps in your own file.

ASK

What it never runs

List the checks and the files it does not touch.

CHANGE

One thing at a time

Change, push, read the run. Repeat.

The supplied workflow is a starting point, not an answer.

Words to search in the Actions docs

Each word is a lever; decide which ones your pipeline needs.

KEYWORDS

Read the reference for each word. Copy nothing you cannot explain, and say in the README why you chose it.

WORKFLOW VOCABULARY

```
on: pull_request, push, branches
jobs, needs
steps: uses, run
permissions
timeout-minutes
concurrency
artifacts: upload, download
```

Pin what you run: a tag can move, a commit SHA cannot.

Cheat sheet: pytest markers and selection

Select what runs, list what would run, stop early.

SELECT TESTS

Markers group tests. With --strict-markers an unregistered marker is an error, and pytest exits with code 5 when nothing is selected.



-m selects by marker, --collect-only lists what would run, -x stops at the first failure.

Cheat sheet: read a run from the terminal

The Checks tab shows the same logs; the command line is optional.

READ A RUN

Find the failing run, read only the failed steps, and save the text. Authenticate gh once; never paste a token into a report.

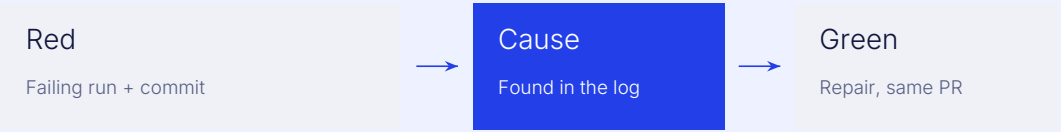
gh (optional)

```
$ gh run list --limit 5
$ gh run view RUN_ID --log-failed
$ gh run view RUN_ID --log
$ gh pr checks
```

Save the log as text in your repo or report; links alone do not last.

Red to green, with evidence

One failing run, one diagnosis, one repair, on the same PR.



A green rerun without a repair is not a repair.

Keep the failing log as text: run records expire, links alone do not last.

Break it on purpose

A teammate adds one believable defect in a PR; the author learns what the tests miss.

CHOOSE

Reviewer picks a bug

Something a teammate could really do; not a fake failure.

EXPECT

A check goes red

If none does, your tests have a gap: say what is missing.

ADD

Add the missing test

Show red, then green, and name the bug it now catches.

The reviewer picks the bug; the author must not know it in advance.

What CI can and cannot claim

Evidence belongs to the machine that produced it.

CAN

Runner-made facts

This commit, installed from the lock on a clean machine, passed these checks.

CANNOT

Laptop facts

Your MLflow runs, local model files, anything the runner did not produce.

WRITE

Three honest lines

In the README, write what CI does and does not show about the model.

Local runs and CI runs are separate claims; cite which one you mean.

Evidence to keep

What a reviewer will look for in your report and repository.

1 **The failing run**

Link, commit and the raw log of the failing job, saved as text.

2 **The repair**

Your diagnosis, the repair commit and the passing run on the same PR.

3 **The map**

A table of your tests: defect, test, and how you know it can fail.

m3 and your individual report

One team checkpoint for Labs 5–6; one personal report.

TEAM

Tag m3

After Labs 5 and 6 are reviewed, submit the link to an annotated tag m3.

PERSONAL

reports/week-3.md

Your own work, one decision, one blocker, any AI help.

RULES

On time, honest

Personal main, no PR or CI. Sunday 23:59. 10% of the course.

Incomplete work is reported as incomplete; never invent a result or a passing check.

EXIT CHECK

Could your pipeline have failed? Show me.

Next, Lecture 7 and Lab 7: turn training into a pipeline with a gate.

Where the examples come from

Condensed from public documentation; read the originals for every option.

REFERENCES

Commands and keywords are condensed for this lab. Check the docs when an option is unclear.



Illustrative examples; run them in your own project before you rely on them.